

Anil Seth



Importing GNUCash Accounts in GNUKhata

gkcore is the REST API core engine of GNUKhata. The GNUKhata app comprises two applications — gkcore and gkwebapp. The objective of this tutorial is to get to know the API.

GNUKhata is an application developed using the Pyramid Web Framework. It comprises two Web applications — a core application called gkcore, and a Web application called gkwebapp. You may easily get started with the installation and development by referring to <https://gitlab.com/gnukhata/gkwebapp/wikis/home>.

As a way of learning how to extend GNUKhata, you may consider importing data from GNUCash into GNUKhata. Since the core and user interface are two separate applications, a good way to learn the core application interface is to create a utility program which will add the GNUCash data.

The utility program will first need to log into the core server, and then issue the commands to add the needed data.

Make sure that you are able to run the core and the Web applications; use the latter to create an organisation and an admin user for the organisation. It is important to keep in mind that the gkcore application needs to be run using the gkadmin user, assuming that you are following the steps from the wiki article; otherwise, it will not be able to access the database.

The login process

You may examine *gkwebapp/views/startup.py* to understand the logic of the steps needed for logging in. The process involves selecting an organisation first, and then supplying the credentials of a user for that organisation.

In order to keep the code as simple as possible, as the objective is to learn the API, select the first organisation. The login credentials are hard-coded. In case of any errors, the utility will just crash and not attempt any error handling.

Once the login is successful, a token is issued. This token will authorise all subsequent calls to the core server. You will notice that the calls to the core server are simple *get* or *post* requests. The data objects transferred between the two are JSON objects.

```
import requests, json
gkhost = 'http://127.0.0.1:6543/'
def getJsonResponse(route, hdrs=None):
    return requests.get(gkhost + route, headers = hdrs).json()

def postJsonResponse(route, jsondata, hdrs=None):
```

```
    return requests.post(gkhost + route, data=jsondata, headers
=hdrs).json()
```

```
def getOrg():
    gkdata = getJsonResponse('organisations')['gkdata']
    first_org = gkdata[0]
    route= ''.join(['orgyears', first_org['orgname'], first_
org['orgtype']])
    gkdata = getJsonResponse(route)['gkdata']
    return gkdata[0]['orgcode']
```

```
def orgLogin(orgcode):
    gkdata = {'username':'anil', 'userpassword':'pswd', 'orgc
ode':orgcode}
    return postJsonResponse('login', json.dumps(gkdata))
['token']
orgcode = getOrg()
gktoken = orgLogin(orgcode)
```

Adding accounts

GNUCash can export the accounts and transactions in CSV files. In the current article, you may extract the accounts into a file, *accounts.csv*. The Python CSV modules make it very easy to handle a CSV file. The first row contains the column labels and should be ignored. You may use the DictReader for more complex processing of the file. For this application, in which only a few columns are needed, the CSV reader is adequate.

There are a few differences in the top level *account/group* names of GNUCash and GNUKhata. So, you need to create a dictionary to map the names from GNUCash to the ones used in GNUKhata.

Some groups in the level below 'Assets' in GNUCash appear as top level groups in GNUKhata, e.g., 'Current Assets' and 'Fixed Assets'. You may ignore 'Assets' from the account hierarchy when transferring the data.

As before, the code below ignores error handling and assumes 'all is well':

```
import csv
def addSubGroup(name, parent, header):
    data = json.dumps({'groupname':name,
'subgroupof':parent})
```